

Einsendeaufgabe MET-E1 – Metriken mit Checkstyle und PMD

1 Einleitung

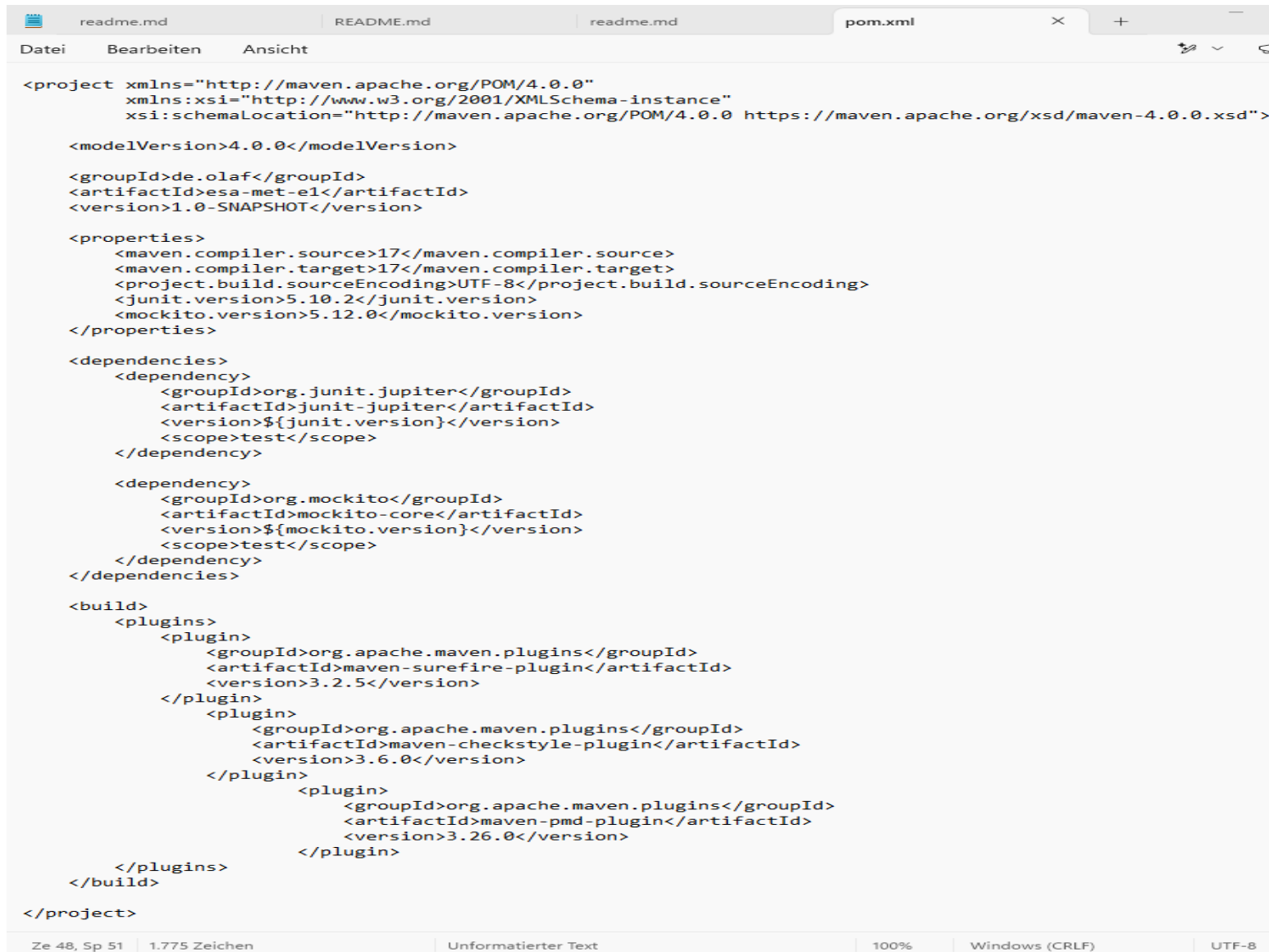
Im Rahmen dieser Einsendeaufgabe wurden zwei Werkzeuge zur statischen Codeanalyse und zur Ermittlung von Softwaremetriken praktisch erprobt. Ziel war es, die Installation und Anwendung der Werkzeuge nachzuvollziehen, deren Ergebnisse zu analysieren und die ausgegebenen Hinweise hinsichtlich ihrer Aussagekraft für die Softwarequalität zu bewerten.

Als Grundlage diente das im Rahmen der vorherigen Einsendeaufgabe (TST) entwickelte Java-Projekt. Dadurch konnten die Analysewerkzeuge unter realistischen Bedingungen getestet und die Ergebnisse anhand eines bereits bekannten Quellcodes interpretiert werden.

Für die Analyse wurden die Werkzeuge **Checkstyle** und **PMD** verwendet, da sie unterschiedliche Aspekte der Codequalität bewerten und sich dadurch sinnvoll ergänzen.

2 Vorbereitung des Maven-Projekts

Zur Durchführung der statischen Codeanalyse wurde das bestehende Maven-Projekt aus der vorherigen Einsendeaufgabe verwendet. Anschließend wurden die Maven-Plugins für **Checkstyle** und **PMD** in die Datei pom.xml eingebunden. Dadurch konnten beide Werkzeuge direkt über Maven ausgeführt werden.



The screenshot shows a code editor with a tab labeled 'pom.xml'. The editor displays the XML configuration for a Maven project. The configuration includes the project's model version, group ID, artifact ID, version, and various properties. It also lists dependencies for JUnit Jupiter and Mockito, and defines build plugins for Maven Surefire, Checkstyle, and PMD. The status bar at the bottom indicates the file is at line 48, column 51, contains 1,775 characters, and is an unformatted text file in UTF-8 encoding.

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>de.olaf</groupId>
  <artifactId>esa-met-el</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <junit.version>5.10.2</junit.version>
    <mockito.version>5.12.0</mockito.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>${junit.version}</version>
      <scope>test</scope>
    </dependency>

    <dependency>
      <groupId>org.mockito</groupId>
      <artifactId>mockito-core</artifactId>
      <version>${mockito.version}</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.2.5</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-checkstyle-plugin</artifactId>
        <version>3.6.0</version>
      </plugin>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-pmd-plugin</artifactId>
        <version>3.26.0</version>
      </plugin>
    </plugins>
  </build>
</project>
```

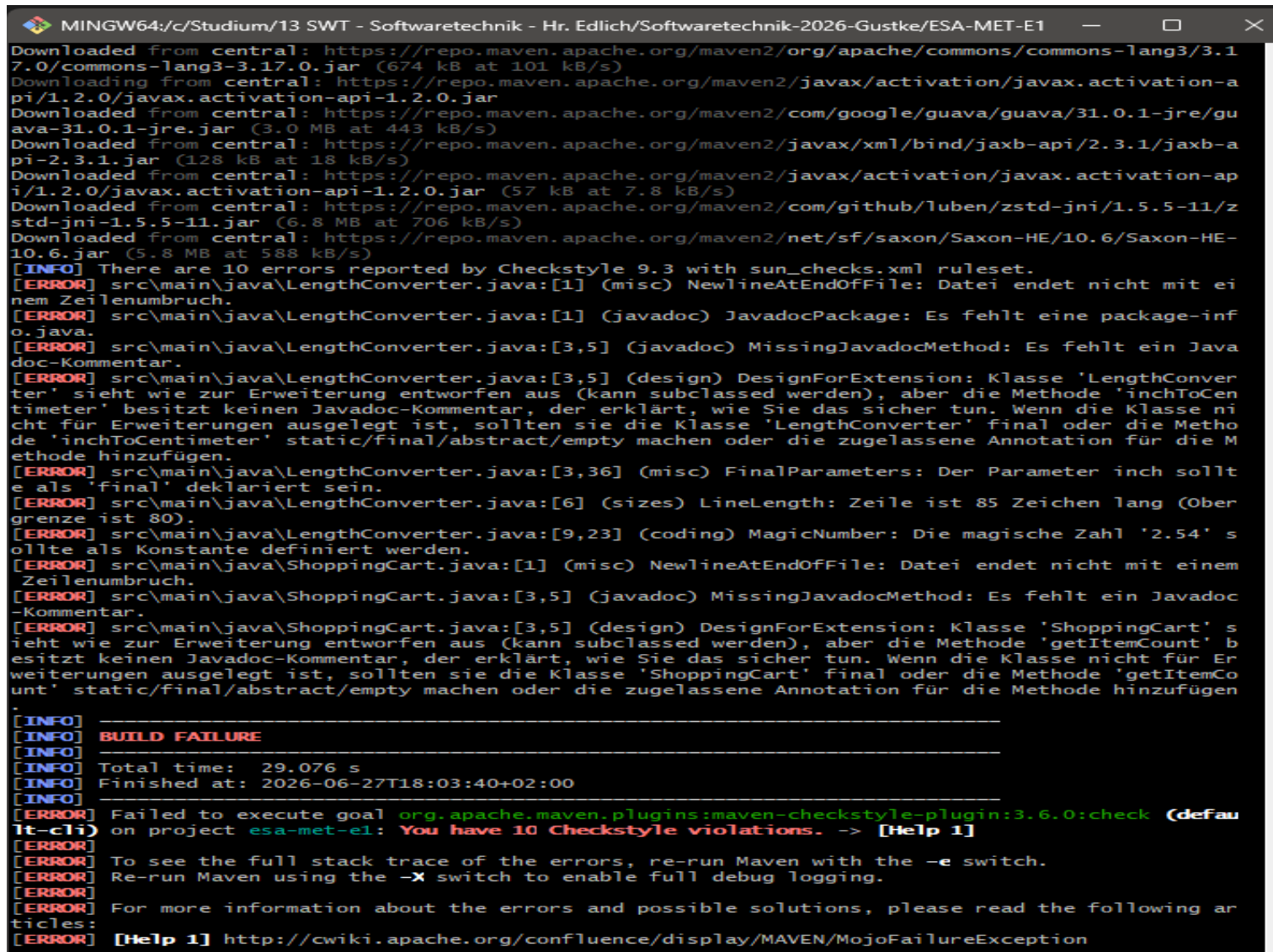
Ze 48, Sp 51 | 1.775 Zeichen | Unformatierter Text | 100% | Windows (CRLF) | UTF-8

Abbildung 1: Konfiguration der Analysewerkzeuge Checkstyle und PMD in der pom.xml

Abbildung 1 zeigt die Konfiguration des Maven-Projekts. Hier wurden die Plugins für **Checkstyle** und **PMD** eingebunden. Durch diese Konfiguration konnten beide Analysewerkzeuge direkt über die Maven-Befehle `mvn checkstyle:check` und `mvn pmd:check` ausgeführt werden.

3 Analyse mit Checkstyle

Nach der erfolgreichen Konfiguration wurde Checkstyle über Maven mit dem Befehl `mvn checkstyle:check` ausgeführt. Ziel war es, Verstöße gegen Programmierkonventionen und definierte Codierungsrichtlinien zu identifizieren.



```
MINGW64/c:/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-MET-E1
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-lang3/3.17.0/commons-lang3-3.17.0.jar (674 kB at 101 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/javax/activation/javax.activation-api/1.2.0/javax.activation-api-1.2.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/com/google/guava/guava/31.0.1-jre/guava-31.0.1-jre.jar (3.0 MB at 443 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/javax/xml/bind/jaxb-api/2.3.1/jaxb-api-2.3.1.jar (128 kB at 18 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/javax/activation/javax.activation-api/1.2.0/javax.activation-api-1.2.0.jar (57 kB at 7.8 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/com/github/luben/zstd-jni/1.5.5-11/zstd-jni-1.5.5-11.jar (6.8 MB at 706 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/net/sf/saxon/Saxon-HE/10.6/Saxon-HE-10.6.jar (5.8 MB at 588 kB/s)
[INFO] There are 10 errors reported by Checkstyle 9.3 with sun_checks.xml ruleset.
[ERROR] src/main/java/LengthConverter.java:[1] (misc) NewlineAtEndOfFile: Datei endet nicht mit einem Zeilenumbruch.
[ERROR] src/main/java/LengthConverter.java:[1] (javadoc) JavadocPackage: Es fehlt eine package-info.java.
[ERROR] src/main/java/LengthConverter.java:[3,5] (javadoc) MissingJavadocMethod: Es fehlt ein Javadoc-Kommentar.
[ERROR] src/main/java/LengthConverter.java:[3,5] (design) DesignForExtension: Klasse 'LengthConverter' sieht wie zur Erweiterung entworfen aus (kann subclassed werden), aber die Methode 'inchToCentimeter' besitzt keinen Javadoc-Kommentar, der erklärt, wie Sie das sicher tun. Wenn die Klasse nicht für Erweiterungen ausgelegt ist, sollten sie die Klasse 'LengthConverter' final oder die Methode 'inchToCentimeter' static/final/abstract/empty machen oder die zugelassene Annotation für die Methode hinzufügen.
[ERROR] src/main/java/LengthConverter.java:[3,36] (misc) FinalParameters: Der Parameter 'inch' sollte als 'final' deklariert sein.
[ERROR] src/main/java/LengthConverter.java:[6] (sizes) LineLength: Zeile ist 85 Zeichen lang (Obergrenze ist 80).
[ERROR] src/main/java/LengthConverter.java:[9,23] (coding) MagicNumber: Die magische Zahl '2.54' sollte als Konstante definiert werden.
[ERROR] src/main/java/ShoppingCart.java:[1] (misc) NewlineAtEndOfFile: Datei endet nicht mit einem Zeilenumbruch.
[ERROR] src/main/java/ShoppingCart.java:[3,5] (javadoc) MissingJavadocMethod: Es fehlt ein Javadoc-Kommentar.
[ERROR] src/main/java/ShoppingCart.java:[3,5] (design) DesignForExtension: Klasse 'ShoppingCart' sieht wie zur Erweiterung entworfen aus (kann subclassed werden), aber die Methode 'getItemCount' besitzt keinen Javadoc-Kommentar, der erklärt, wie Sie das sicher tun. Wenn die Klasse nicht für Erweiterungen ausgelegt ist, sollten sie die Klasse 'ShoppingCart' final oder die Methode 'getItemCount' static/final/abstract/empty machen oder die zugelassene Annotation für die Methode hinzufügen.
[INFO]
[INFO] BUILD FAILURE
[INFO]
[INFO] Total time: 29.076 s
[INFO] Finished at: 2026-06-27T18:03:40+02:00
[INFO]
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-checkstyle-plugin:3.6.0:check (default-cli) on project esa-met-e1: You have 10 Checkstyle violations. -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
```

Abbildung 2: Ergebnis der statischen Codeanalyse mit Checkstyle (10 Regelverstöße)

4 Analyse mit PMD

Im Anschluss wurde PMD mit dem Maven-Befehl `mvn pmd:check` ausgeführt. Im Gegensatz zu Checkstyle konzentriert sich PMD stärker auf potenzielle Programmierfehler, ineffiziente Konstruktionen und sogenannte *Code Smells*.

```
Downloaded from central: https://repo.maven.apache.org/maven2/net/sourceforge/pmd/pmd-java/7.7.0/pmd-
java-7.7.0.jar (1.7 MB at 333 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/net/sf/saxon/Saxon-HE/12.5/Saxon-HE-12
.5.jar (5.6 MB at 835 kB/s)
[WARNING] Unable to locate Source XRef to link to -- DISABLED
[INFO] PMD version: 7.7.0
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/skins/maven-fluido-s
kin/2.0.0-M9/maven-fluido-skin-2.0.0-M9.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/skins/maven-fluido-sk
in/2.0.0-M9/maven-fluido-skin-2.0.0-M9.jar (355 kB at 1.7 MB/s)
[INFO] Rendering content with org.apache.maven.skins:maven-fluido-skin:jar:2.0.0-M9 skin
[INFO]
[INFO] <<< pmd:3.26.0:check (default-cli) < :pmd @ esa-met-e1 <<<
[INFO]
[INFO]
[INFO] — pmd:3.26.0:check (default-cli) @ esa-met-e1 —
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 20.788 s
[INFO] Finished at: 2026-06-27T18:11:00+02:00
[INFO]
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-G
ustke/ESA-MET-E1 (master)
$
```

Abbildung 3: Erfolgreiche Ausführung der statischen Codeanalyse mit PMD (BUILD SUCCESS)

5 Vergleich der Werkzeuge

Checkstyle und PMD dienen beide der statischen Codeanalyse, verfolgen jedoch unterschiedliche Schwerpunkte. Checkstyle überprüft hauptsächlich die Einhaltung von Programmierkonventionen und Codierungsrichtlinien, beispielsweise hinsichtlich Dokumentation, Formatierung oder Namenskonventionen. PMD konzentriert sich dagegen stärker auf potenzielle Programmierfehler, ineffiziente Konstruktionen und Code-Smells. Während Checkstyle im untersuchten Projekt zehn Regelverstöße identifizierte und den Build mit **BUILD FAILURE** beendete, verlief die Analyse mit PMD erfolgreich (**BUILD SUCCESS**). Beide Werkzeuge ergänzen sich und tragen gemeinsam zur Verbesserung der Softwarequalität bei.

6 Fazit

Im Rahmen dieser Einsendaufgabe wurden die Werkzeuge Checkstyle und PMD erfolgreich in ein Maven-Projekt integriert und praktisch angewendet. Beide Werkzeuge konnten ohne größeren Installationsaufwand genutzt werden und lieferten aussagekräftige Ergebnisse zur Bewertung der Codequalität. Während Checkstyle mehrere Verstöße gegen Programmierkonventionen identifizierte, zeigte PMD keine kritischen Auffälligkeiten. Die praktische Anwendung verdeutlicht, dass beide Werkzeuge unterschiedliche Aspekte der Softwarequalität analysieren und sich sinnvoll ergänzen.

Die praktische Durchführung der Analysen hat gezeigt, dass unterschiedliche Werkzeuge verschiedene Qualitätsaspekte des Quellcodes bewerten und sich dadurch sinnvoll ergänzen.

7 Quellen

Apache Maven Project. (2025). Maven Checkstyle Plugin.

<https://maven.apache.org/plugins/maven-checkstyle-plugin/>

Apache Maven Project. (2025). Maven PMD Plugin.

<https://maven.apache.org/plugins/maven-pmd-plugin/>

Checkstyle Team. (2025). Checkstyle.

<https://checkstyle.sourceforge.io/>

PMD Open Source Project. (2025). PMD Source Code Analyzer.

<https://pmd.github.io/>